COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	July 17, 2026	**DATE SUBMITTED:**	July 22, 2026
NAME:	Uchi, Mao Sophie S.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2 **ARRAYS, POINTERS AND DYNAMIC MEMORY ALLOCATION**

1. PROCEDURE OUTPUT

Table 2-1. Initial Driver Program

Screenshot

main.cpp

HOA3 > main.cpp > ...

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
    //constructor
    Student(std::string newName ="John Doe", int newAge=18){
    studentName = std::move(newName);
    studentAge = newAge;
    std::cout << "Constructor Called." << std::endl;
};

    //deconstructor
    ~Student(){
    std::cout << "Destructor Called." << std::endl;
    }
```

```
21
22      //Copy Constructor
23      Student(const Student &copyStudent){
24      std::cout << "Copy Constructor Called" << std::endl;
25      studentName = copyStudent.studentName;
26      studentAge = copyStudent.studentAge;
27      }
28
29      //Display Attributes
30      void printDetails(){
31      std::cout << this->studentName << " " << this->studentAge << std::endl;
32      }
33  };
34
35  int main() {
36      Student student1("Roman", 28);
37      Student student2(student1);
38      Student student3;
39      student3 = student2;
40
41      return 0;
42  }
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S4-UCHI>
```

Observation	Excluding explaining each four main operators, I'll be explaining what was shown at our output. Line 36 shows a Constructor is called to create a student, but a student should at least have a name and an age, parameterizing object student1. Line 37 shows a Copy Constructor is called to create another student, with the existing student1's data, simply duplicating the previous object with the same name and age in the new object. Line 38 shows a Constructor is called but nothing to parameterize, instead it automatically displays the preset name and age from the Constructor function. Line 39 was supposed to call for Copy Assignment, but wasn't implemented as it's supposed to safely assign student2's value to student3 without creating a new object created from this line. The remaining 3 outputs called for the Destructor constructor, deleting all 3 object's data that was created, but not the object itself.

Table 2-2. Modified Driver Program with Student Lists

Screenshot

```
35  int main() {
36      const size_t j = 5;
37
38      Student studentList[j] = {};
39      std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
40      int ageList[j] = {15, 16, 18, 19, 16};
41      return 0;
42  }
```

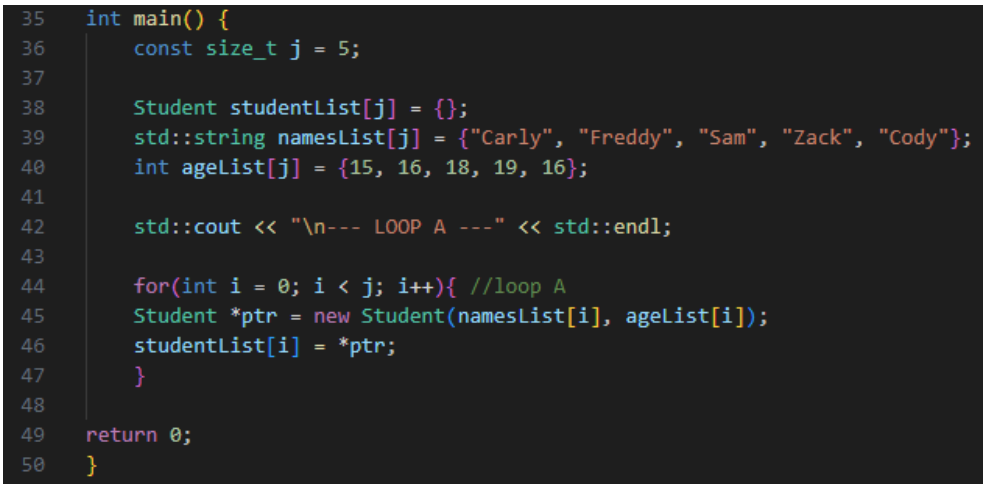
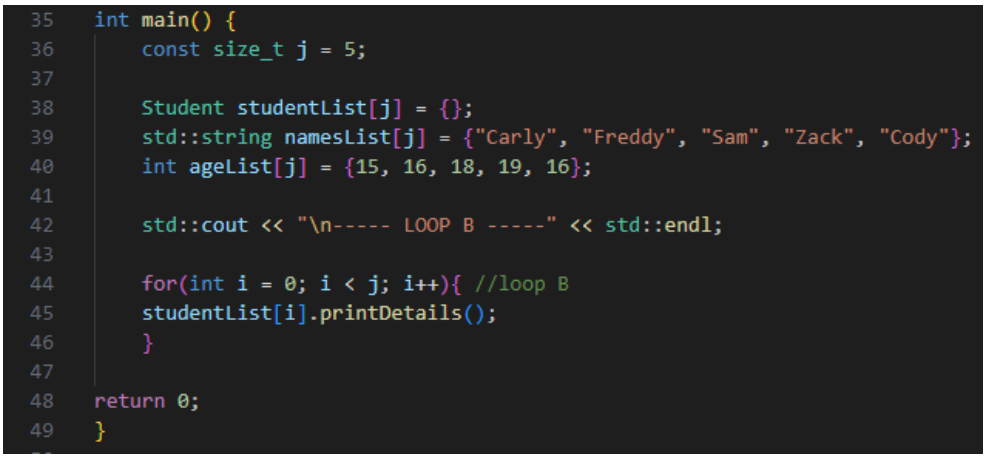
```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S4-UCHI>
```

Observation

After creating a size for 5 student objects, those 5 objects are now in the class Student's studentlist array, so initializing them would only require one line. Then the code called for the Constructor operator 5 times to input data to fill up the space for the time being and has been initialized with the preset name and age. Afterward, at line 39, individual names will be parameterized, soon will overwrite the preset name from the student object. Similarly to line 40, individual age will also be parameterized, soon will overwrite the preset age from the student object. For the result, line 39 and line 40 overtook the preset name and age with new data, but the object itself has not changed nor duplicated. Returning our code, all of the student object data will be removed, and an empty student object remains.

Table 2-3. Final Driver Program

Loop A	
Observation	Skipping the previous Student list driver program, we will now move on to Loop A that contains For Loop when everytime you've attempted to use new object, you create a new Student object on the heap, while also copying the preset name and age from the array list (nameList and ageList).
Loop B	
Observation	Skipping the previous Student list driver program, we will now move on to Loop B that also contains For Loop, when it starts, every student at the index "int i" will continuously repeat to print the details of the studentList with the preset name and age first, then it will automatically replace the data name and age using the ".printDetails", which contains the nameList and ageList.
Output	LOOP A

```
35  int main() {
36      const size_t j = 5;
37
38      Student studentList[j] = {};
39      std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
40      int ageList[j] = {15, 16, 18, 19, 16};
41
42      std::cout << "\n--- LOOP A ---" << std::endl;
43
44      for(int i = 0; i < j; i++){ //loop A
45      Student *ptr = new Student(namesList[i], ageList[i]);
46      studentList[i] = *ptr;
47      }
48
49  return 0;
50  }
```

```
35  int main() {
36      const size_t j = 5;
37
38      Student studentList[j] = {};
39      std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
40      int ageList[j] = {15, 16, 18, 19, 16};
41
42      std::cout << "\n----- LOOP B -----" << std::endl;
43
44      for(int i = 0; i < j; i++){ //loop B
45      studentList[i].printDetails();
46      }
47
48  return 0;
49  }
```

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.

----- LOOP A -----
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S4-UCHI>
```

LOOP B

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.

----- LOOP B -----
John Doe 18
John Doe 18
John Doe 18
John Doe 18
John Doe 18
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S4-UCHI>
```

Observation	For me, there isn't much of a difference, but the way you create your new object and replace the data would be very different because your loop starts differently. Loop A would create

	the object with nothing in it at first, then add the data later, whereas Loop B would also create the object WITH the preset data; instead of adding new data, you would only need to replace the data. However, both return as empty objects, so depending on your preferences, either code may work.

Table 2-4. Modifications/Corrections Necessary	
Modification	N/A
Observation	N/A

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers
Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

ilo_a | ilo_c.cpp 9+

ilo_c.cpp > ...

```cpp
#include <iostream>
#include <string>

class Fruits{
private:
    std::string* fruit;
    double price;
    int quantity;

public:
    // Constructor
    Fruits(std::string newFruit, int priceF, int quantityF){
        fruit = std::move(newFruit);
        price = priceF;
        quantity = quantityF;
    }

    // Destructor
    ~Fruits(){
    }

    // Copy Assignment Operator
    Fruits& operator=(const Fruits& copy){
        if(this != &copy){
            fruit = copy.fruit;
            price = copy.price;
            quantity = copy.quantity;
        }
        return *this;
    }
```

ilo_a ilo_c.cpp 9+

ilo_c.cpp > ...

```cpp
class Fruits{
    Fruits& operator=(const Fruits& copy){

    // Copy Constructor
    Fruits(const Fruits& copy){
        fruit = copy.fruit;
        price = copy.price;
        quantity = copy.quantity;
    }
    void displayAttributes() const{
        std::cout << fruit << ":" << " PHP " << price << " x" << quantity << std::endl;
    }

    int getPrice() const{
        return price;
    }

    int getQuantity() const{
        return quantity;
    }
};

class Vegetables{
private:
    std::string* vegetables;
    double price;
    int quantity;

public:
    // Constructor
    Vegetables(std::string newVegetables, int priceV, int quantityV){
        vegetables = std::move(newVegetables);
```

```
61          price = priceV;
62          quantity = quantityV;
63      }
64
65      // Destructor
66          ~Vegetables(){
67          }
68
69      // Copy Constructor
70      Vegetables(const Vegetables& copy){
71          vegetables = copy.vegetables;
72          price = copy.price;
73          quantity = copy.quantity;
74      }
75
76      // Copy Assignment Operator
77      Vegetables& operator=(const Vegetables& copy){
78          if(this != &copy){
79              vegetables = copy.vegetables;
80              price = copy.price;
81              quantity = copy.quantity;
82          }
83          return *this;
84      }
85
86      void displayAttributes() const{
87          std::cout << vegetables << ":" << " PHP " << price << " x" << quantity << std::endl;
88      }
89
90      int getPrice() const{
```

```
        return price;
    }

    int getQuantity() const{
        return quantity;
    }
};

// Total Sum Function
void totalSum(Fruits *fruits[], int fruitzSize,
              Vegetables *vegetables[], int vegetablesSize)
{
    int total = 0;

    for(int i = 0; i < fruitzSize; i++){
        if(fruits[i] != nullptr){
            total += fruit[i]->getPrice() * fruit[i]->getQuantity();
        }
    }

    for(int i = 0; i < vegetablesSize; i++){
        if(vegetables[i] != nullptr){
            total += vegetables[i]->getPrice() * vegetables[i]->getQuantity();
        }
    }

    std::cout << "\nTotal Sum: PHP " << total << std::endl;
}

int main(){

```

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

```
120  int main(){
121
122      const int sizeF = 3, sizeV = 3;
123
124      Fruit *fruit[sizeF];
125      Vegetable *vegetable[sizeV];
126
127      fruit[0] = new Fruit("Orange", 70, 6);
128      fruit[1] = new Fruit("Papaya", 150, 7);
129      fruit[2] = new Fruit("Watermelon", 250, 8);
130
131      vegetable[0] = new Vegetable("Ampalaya", 180, 8);
132      vegetable[1] = new Vegetable("Cucumber", 140, 9);
133      vegetable[2] = new Vegetable("Lettuce", 150, 10);
134
135      std::cout << "Jenna's Grocery List" << std::endl;
```

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

```
99   // Total Sum Function
100  void totalSum(Fruits *fruits[], int fruitzSize,
101                Vegetables *vegetables[], int vegetablesSize)
102  {
103      int total = 0;
104
105      for(int i = 0; i < fruitzSize; i++){
106          if(fruits[i] != nullptr){
107              total += fruit[i]->getPrice() * fruit[i]->getQuantity();
108          }
109      }
110
111      for(int i = 0; i < vegetablesSize; i++){
112          if(vegetables[i] != nullptr){
113              total += vegetables[i]->getPrice() * vegetables[i]->getQuantity();
114          }
115      }
116
117      std::cout << "\nTotal Sum: PHP " << total << std::endl;
118  }
```

```
137     for(int i = 0; i < sizeF; i++){
138         fruit[i]->displayAttributes();
139     }
140
141     for(int i = 0; i < sizeV; i++){
142         vegetable[i]->displayAttributes();
143     }
144
145     totalSum(fruit, sizeF, vegetable, sizeV);
```

```
 99  // Total Sum Function
100  void totalSum(Fruits *fruits[], int fruitzSize,
101                Vegetables *vegetables[], int vegetablesSize)
102  {
103      int total = 0;
104
105      for(int i = 0; i < fruitzSize; i++){
106          if(fruits[i] != nullptr){
107              total += fruit[i]->getPrice() * fruit[i]->getQuantity();
108          }
109      }
110
111      for(int i = 0; i < vegetablesSize; i++){
112          if(vegetables[i] != nullptr){
113              total += vegetables[i]->getPrice() * vegetables[i]->getQuantity();
114          }
115      }
116
117      std::cout << "\nTotal Sum: PHP " << total << std::endl;
118  }
```

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

```
147     // Problem 4
148     delete vegetable[2];
149     vegetable[2] = nullptr;
150
151     for(int i = 0; i < sizeF; i++){
152         delete fruit[i];
153         fruit[i] = nullptr;
154     }
155
156     delete vegetable[0];
157     vegetable[0] = nullptr;
158
159     return 0;
160 }
```

Conclusion

What I've learned from this activity is I've realized on how I should use the four operators, and its function is very different, yet it still confuses me from time to time, but if I searched on what was the difference, I would get it, but now that I realize its pretty simple to remember if you think of the specific function it has for example deconstructor uses ~ or the NOT operator, came from the boolean library to delete a specific data in your object. Also, including the initializing array in this activity helps me very much as I would be scared if I had to use these operators line by line. With these operators, it will be easier for me to use it in my future codes for simplicity and readability.

The sad part of this activity is I could not get the correct code for Supplementary questions, as I kept getting errors again and again, but I would have to give up on this one as many activities day by day has actually taken a toll on me and might not properly execute on what I've wanted to do with my code, but surely in the future, if I truly have enough time to create codes this difficult, I shall learn it, even by little bit is what I acknowledge from this lesson, then so be it, all I had to do is work harder in the future.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: *Google AI Overview, QuillBot*

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: *Code Assistance, Explanation of Concepts, Grammar Correction*

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: *After trial and errors, I succumb to check on what's wrong with my code with AI Assistance and making sure I understood what went wrong and what fixed it, but alas I can't finish it on time as many errors are left unfixed.*

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.